Influence Maximization Analysis; Progress Report

Antoni Zyla Supervised by Nathan Griffiths

Contents

1 Introduction **2**

2 Background Research **2**
2.1 Brief Literature Review . . . 2
2.2 Models . . . 2

3 Current Status **2**
3.1 Working with Networks . . . 3
3.2 Synthetic Networks . . . 3
3.3 Cascade and Threshold Models . . . 4
3.4 Evaluation of Sampling Methods . . . 6
3.5 Packaging . . . 6
3.6 Parallelisation . . . 7
3.7 Documentation . . . 7
3.8 Technical Content . . . 7
3.8.1 Code Structure . . . 8
3.9 Performance . . . 8

4 Upcoming Goals **8**
4.1 Areas I will Investigate . . . 8

5 Reflection **8**

6 Project Management **8**

7 Legal, Social, Ethical and Professional Considerations **8**

A Abstract **8**

B Problem Discussion **9**
B.1 Models . . . 9
B.2 Sampling Methods . . . 9
B.3 Heuristics . . . 10
B.4 Application to research . . . 11

C Objectives **11**

D Methods and Methodology **12**

E Timeline **12**

F Resources and Risks **12**
F.1 Resources 12
F.2 Risks 13

G Legal, Social, Ethical and Professional Considerations **13**

1 Introduction

Social networks [**?**], interactions [**?**] and email chains [**?**] can all be modelled as a graph in the $G = (V, E)$ sense.

Just as in a social setting concepts or ideas can spread through networks, for example if someone in a friend group takes up a new hobby then they have a higher chance of influencing their closest friends to try it than someone who they don't know. This idea leads directly into the independent cascade model (ICC), each node is able to infect every one of it's direct connections once with probability p. Those that it infects are then able to continue the cascade until there are no more active nodes.

An extension of the ICC model is the decreasing cascade model (DCC), wherein the probability of a node being 'turned' by it's neighbors decreases each time it is attempted. If you see an advertisement for a certain product a very large amount of time you may be less likely to purchase or use the product due to overstauration/annoyyance. The rate at which we decrease the probability of infection is defined by a function $f(G, n) = p \in [0, 1]$.

Another way of considering these problems is to consider a model where a node may adopt a concept after a certain proportion of it's neighbours have it. This models things such as immunity from diseases quite well. Each node is given it's own $t \in T$ where $T \sim U(0, 1)$, once the node's proportion of infected neighbours reach t then the node becomes infected.

2 Background Research

I have conducted my research in this topic from a wide range of sources. Primarily in the form of articles about the research area.

2.1 Brief Literature Review

2.2 Models

First introduced in 2003 by [**?**] the problem of influence maximization in social networks has been an active research area since it's introduction. Independent cascade and linear threshold models were introduced and a base greedy algorithm was introduced.

3 Current Status

Overall good progress has been made, a good portion of the desired objectives have been achieved. The codebase is in average shape, documentation from docstrings is automatically generated into a navigable website that links between functions classes with examples of how use each function.

Overall as from 1 the project is in good shape to achieve all objectives by the end of term 2.

Objective	Status	Notes
1. Import and Export of Graphs	Complete	Not implemented unit testing
2. Generation of Synthetic Graphs	Complete	
3. Influence Models	Complete	
4. Sampling Methods	Partially Complete	TODO: Albatross Sampling
5a. Cost based analysis heuristics	Incomplete	Would be the most complex, may not implement if not needed
5b. Minimum Nodes heuristics	Incomplete	
5c. Rate vs Absolute change heuristics	Partially Complete	TODO: Rate Analysis
6. Packaging	Complete	
7. Parallelisation	Complete	
8. Documentation	Partially Complete	

Table 1: Current Status Summarised

3.1 Working with Networks

As listed in objective 1 my project needs the ability to import and export networks. I have implemented the ability to import networks from a file as commonly found online [], custom delimiters are able to be used. To make getting started with my program quick and easy I have included some networks inside of the python package, they are listed below in 2 along with some information about them.

Name	Nodes	Edges	Mean Deg.	Cluster. Coef.	Diam.	Centralization
Enron	36692	183831	10.020	0.497	13	
P2P	10876	39994	7.355	0.006		
Airports	1574	17215	21.874	0.504		

Table 2: Statistics on Included Networks with my package, to 3 d.p. when applicable.

3.2 Synthetic Networks

Some papers ([4] and []) commonly compare synthetic networks to sampled networks from realworld structures. As part of Objective 2 I have implemented the creation of both Kleinberg and Eppstein generators. Examples of some networks produced by them are listed in the table below.

Name	Nodes	Edges	Mean Deg.	Clust. Coef.	Diam.	Centralization
Kleinberg1000	961	4486	9.336	0.052	14	-0.0066
Kleinberg5000	4900	23390	9.547	0.0385	16	-0.0013
Eppstein1000						
Eppstein5000						

Table 3: Statistics on example synthetic networks with my package, to 3 d.p. when applicable.

The data in both table 3 and table 2 was easily generated in just a few lines of code and outputted a formatted table in the form of a pandas data frame as demonstrated below in

the code for table 3.

Listing 1: Code to generate table 3

```
nets = [
    im.Network(name="Kleinberg1000"),
    im.Network(name="Kleinberg5000"),
    im.Network(name="Eppstein1000"),
    im.Network(name="Eppstein5000"),
]

nets[0].kleinbergs_generator(100)
nets[1].kleinbergs_generator(500)
nets[2].eppstein_generator(1000)
nets[3].eppstein_generator(5000)

rows = [Metrics.generate_tabular_row(net) for net in nets]
table = pd.concat(rows)
```

3.3 Cascade and Threshold Models

As written in point 3, Independent Cascade, Decreasing Cascade and Linear Threshold models have been implemented. Code for automated running of trials, testing of heuristics on each of these has been written. The required computations perform well when ran on the department's compute cluster. Data is output in both tabular and graphical formats, below is an example of some early runs.

There are a few things we can garner from figure 1:

1. Degree based heuristics are highly correlated to the spread of a set of seed nodes, in relation to their computational cost.
2. Clustering Coefficient is very effective, however only after a certain number of seeds, this could be due to how clustering and degree are presented, a high degree node may be connected to a significant proportion of the network whereas a node with a high clustering coefficient doesn't necessarily have a high possible number of effected neighbors.
3. The asymptotic behavior of degree based heuristics shows that the network topology is being respected by the MHDA sampling in regards to the distribution of node degree.
4. There being minimal spread beyond the based starting nodes in the decreasing cascade and a convergence of the heuristics demonstrates that there may need to be some adjustment to the loss in turning a node per attempt.
5. The linear threshold model shows a similar pattern to the independent cascade where the heuristics are partitioned into two groups, naive and slight more complex. ..

Following figure 1 I implemented a few more heuristics as well as the naive greedy algorithm and the improved celf version.

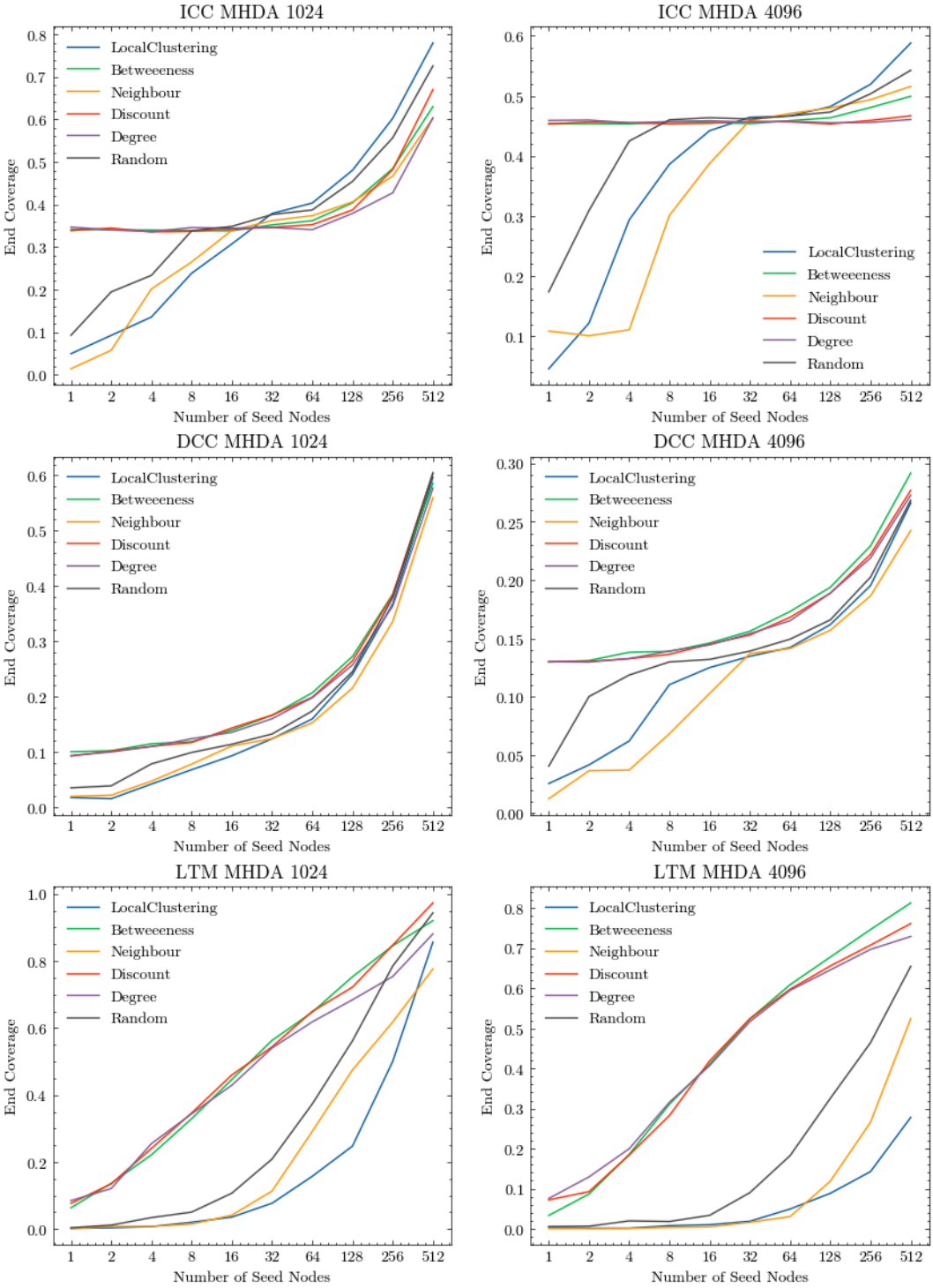


Figure 1: Examples of Early Running of Models with heuristics

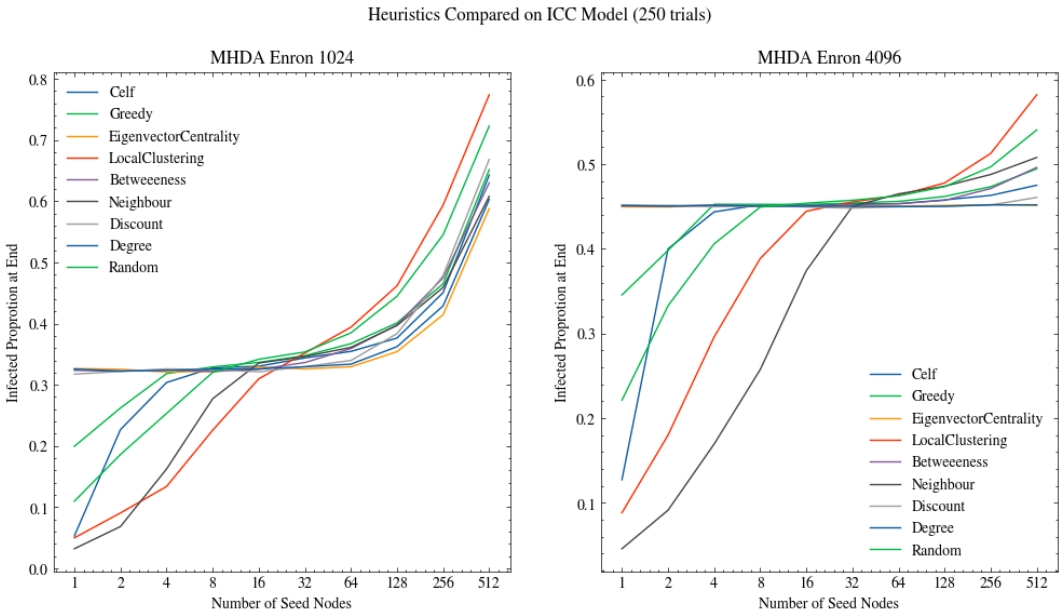


Figure 2: Currently Implemented Heurstics on ICC

3.4 Evaluation of Sampling Methods

As written in section B.2 of the specification, sampling methods 1 to 5 have been implemented. Running multiple samples, collecting and collating the results has been written. Tables similar to 2 or 3 are output with data about their mean and standard deviations.

3.5 Packaging

I am utilising an open source python library packaging tool called Hatchling, allowing me to package the entirety of the code into a `.tar.gz` and a `.whl` distributable. Python3.10 and above is required due to the introduction of union types in the type annotation system. I have attempted to use PyPy (the JIT compiling interpreter) however there were some issues with the multiprocessing code so I have decided to not focus on this aspect.

After the project has been packaged it's trivial to install and use on any system with a recent python installation.

Listing 2: Utilising my code

```
shell> pip install Influence_Modelling.whl

from Infleunce_Modelling import Network as im
import networkx as nx

net = im.Network("test-network")
net.set_nx(nx.complete_graph(10))
iters, final = net.run_icc()
```

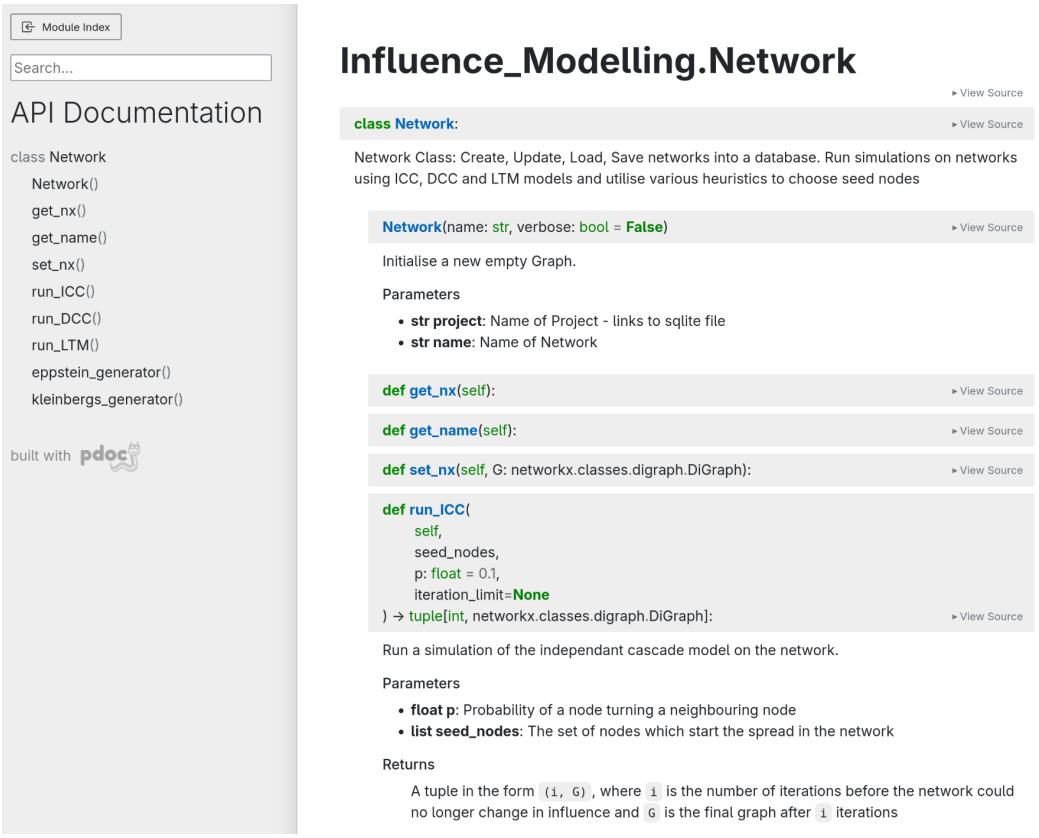


Figure 3: Example of automated documentation site

3.6 Parallelisation

Running trials and generating samples of networks has been implemented using pools and python's multiprocessing library. Performance is good when it's shared across a single node, splitting across multiple physical servers resulted in a minor speedup at the expense of much higher communication cost.

This is not a large problem as I can split my running of the testing into sepearate invocations of my scripts on the different nodes.

3.7 Documentation

As part of the projects packaging phase automatic documentation is generated from the usage of python docstrings, this is done using an opensource library called pdoc [1].

Using github actions this build steps runs on every commit to the main branch of the repository and this generates a navigable and searchable documentation base as shown in figure 3.

3.8 Technical Content

Talk about code structure, data collection, processing of data, saving intermediate steps in all computations, using the hpc cluster, typical performance of runs.

[1] https://pdoc.dev/

3.8.1 Code Structure

The primary code is structured in an object oriented fashion, for example a network is a class which is then extended into a network that reads or writes csv files.

Each of various sections I need to use are split into multiple files and these all have to be imported separately when being used.

Most of the code is used in interactive notebook environments.

My test scripts are standalone scripts that simply require the packaged library to be installed alongside some other libraries, these are the scripts which produce the results as displayed in all the tables and figures so far in this document.

3.9 Performance

For every run of a cascade, sampling method or any action I record the time taken, if running overa large number of samples I also automatically take the mean and standard deviation of the running time and this is an extra column at the end of each of the tabular outputs for any aggregate process.

4 Upcoming Goals

Talk about being on track with the goals from the gannt chart in spec, need to implement some more stuff and actually do some independent testing and evaluation of my own ideas.

4.1 Areas I will Investigate

An iterative approach based on a combination of greedy and heuristic methods. For example choose seeds by degree, redo degree but don't include any nodes that that node has spread to over a few iterations of the cascade model.

Combinations of some heuristics such as clustering coefficient and betweenness.

5 Reflection

6 Project Management

Overall as shown through this document so far progress is on track with the proposed timelines. Software and technology used has not changed, there has not been a large shift in goals or requirements.

7 Legal, Social, Ethical and Professional Considerations

As per section G of the specification there have been no changes to this.

Appendix: Project Specification

A Abstract

A network is a weakly connected directed graph about which we reason. Influence is the measure of how ideas or properties spread through this graph based on certain models; choosing the seed set of nodes from which to begin the model determines how quickly and

how far the ideas will propagate. Social networks [10], infectious diseases and behaviors [1] can be studied by using such an approach and can be used to inform decision making in many areas.

Finding a subset of nodes of the network which produce the greatest influence spread after a given time period could be used to determine those who'd best promote a product or be at greatest risk of spreading an infectious disease [1]. Finding this subset is difficult and known to be NP-Hard, as a reduction from Vertex Cover [7]. Using certain heuristics such as the degree of a node to choose the seed nodes is significantly more efficient running in polynomial time [3].

Using real world networks to perform this analysis on is not only difficult to obtain but also very large and computationally intensive [9]. I will aim to create a tool which can be used to evaluate and compare different heuristics and sampling methodologies against different types of networks. I will then use this tool to find a more representative sampling method on networks, or a heuristic which works more effectively on a certain set of topologies.

B Problem Discussion

I will define a network upon which the models below will work on as a weakly connected directed graph. Doing so allows for modelling of networks in which the relationships are bidirectional (undirected) by simply creating two edges to represent that relationship, and for modelling directed networks such as email chains that have a clear sender and recipient. To be used for analysis, the network needs to be connected, as an idea cannot directly propagate between two disjoint components.

B.1 Models

1. Independent Cascade Model (ICM) - a node can be 'active' or 'inactive' and has a constant probability of infecting each of it's neighbours. Every infectious node has one attempt to infect it's neighbours before becoming inactive [7].

2. Decreasing Cascade Model (DCM) - an extension of the IC model the probability of infecting a node is determined by previous attempts, modelling how subsequent attempts are less likely to be successful after failed attempts [**?**].

3. Linear Threshold Model (LTM) - A model where each node has a certain 'threshold' required to infect it, considering the cumulative effect of the already infected nodes on the node determines if it will become infected [7].

B.2 Sampling Methods

Some real world datasets are far too large to efficiently compute these models on. Sampling these networks allows us to find a smaller network that encapsulates their features but is much more efficient to perform computations on. Those listed below I have researched and feel would be able to implement and use to evaluate against one another, as research progresses I may choose to add more methods or to not use a certain method.

1. Breadth First Search (BFS) - A natural exploration of the graph from a random node in layers [4].

2. Snowball Sample (SNS) - An extension of BFS which will stop once the desired sample size is reached [4].

3. Random Walk (RW) - Starting at a random node in the network chooses where to go onwards based on the the node's neighbours being chosen at random.

4. Metropolis Hastings Random Walk (MHRW) - A random walk with a modification to the probability function used to determine the subsequent node. This is to generate a uniform distribution of node degrees by favouring lower degree nodes [5].

5. Metropolis Hastings Random Walk with Delayed Acceptance (MHDA) - A further modification of MHRW in which the probability of backtracking to the previous node is reduced [8].

6. Albatross Sampling (AS) - A random walk with a chance of jumping to other locations in the network to find connected components [6].

B.3 Heuristics

Choosing which nodes to start the models on will affect how quickly the influence will propagate and how far it will spread to. A greedy approach may be close to optimal, however on larger datasets is inefficient and potentially intractable [2]. The following are some of the methods by which I will investigate, during the research phase I may add more or I may exclude some.

1. Random - As a baseline choose k random nodes.

2. Degree - Choose nodes with the highest degree.

3. Degree Discount - A modification of the degree heuristic but with a lower chance of choosing a node if it's neighbour is in the seed set [3].

4. Local Clustering Coefficient - The extent to which neighbours are connected to one another [4].

5. Average Neighbour Degree - Measuring the average degree of a node's neighbours may allow for communicating with high influence nodes [4].

6. Betweenness Centrality - Measuring how many shortests path's in the network pass throuhg a point [4].

7. Average Shortest Path Length - A measure of centrality by taking the mean length of the shortest path from the node to every other node [4].

8. Edge Embeddedness - For an edge (u, v) is the size of the intersection of the neighbourhoods of both u and v [4]. This serves as a measure of how tightly packed a network is around a certain area.

9. Edge Overlap - For an edge (u, v) the edge overlap is the ratio of it's embeddedness to the size of the union of the neighbourhoods of u and v. This can be used to determine the 'location' of an edge, for example if it connects connected components it may have a low overlap whilst a very high embeddedness may indicate not only is the agent well connected inside it's cluster it's also connected to other clusters [4].

10. Traditional Greedy Algorithm (Select Top k in [10]) - Choose greedily from nodes which have the highest influence [7]. This is inefficient due to requiring running a cascade for each node. It approaches within $(1 - 1/e)$ of the optimal solution so will be used to compare against heuristics.

B.4 Application to research

Utilising the above concepts, possible pieces of research in this area may include:

- Finding a more effective or representative heuristic for finding seed nodes in a network.
- Optimising a heuristic's running time whilst maintaining performance in regards to representation.
- Finding a more representative sampling method for varied network topologies.
- Finding a correlation between certain topological features and the effectiveness of one sampling method or heuristic over another, particularly in relation to that network feature.

C Objectives

The exact goal of my project is not fully decided, however there are some requirements for the code. With these features I would be able to perform much of the computation and comparison of various papers. Ideally I would be able to find something new or improve on existing techniques as described in section B.4.

1. Facilitate import and export of different graph formats.
2. Generation of synthetic graphs using graph generators for consistent analysis when real world datasets could be too large to run quick computations on.
3. Running and evaluation of the Independent Cascade Model, Decreasing Cascade Model and Linear Threshold Model. Allowing for evaluation of different heuristics against these models on different graphs.
4. Creation and evaluation of new sampling methods.
    - Evaluate the running time of sampling algorithms in practice, finding the real world constant factors.
    - Allow for evaluation of influence spread on the same network with different samples and how they affect it.
5. Creation and evaluation of new heuristics for selection of seed nodes against benchmark algorithms.
    - A 'cost' based strategy could be to give each algorithm a total cost, and each node has a value with the aim to find the subset that produces the greatest spread from nodes within a certain total 'cost'.
    - A 'minimum nodes' strategy to meet some spread metric, for example finding the lowest number of nodes that will produce a $\delta\%$ spread after x iterations.
    - Investigation of heuristics focused on overall growth in influence or rate of change in influence - this would involve charting the growth rate of various heuristics against eachother with trendlines etc.
6. Python **whl** and **tarball** packaging to allow for easy installation on various systems.
7. Programming with parallelisation in mind. Some results may take hours to compute on a personal device but are able to be ran in parallel on a cluster or more powerful system. I will attempt to use the batch compute system to produce any results.
8. Fully document all code, usage examples, parallelisation examples, and everything required to produce any results in my final report.

D Methods and Methodology

The project will be broken up into three main stages; research, implementation, and evaluation. During each stage I will be in contact with my supervisor to ensure that I am on the right track and to keep me accountable.

1. The research stage will require a detailed literature review of articles and papers pertaining to network influence modelling, methods, heuristics and sampling. I will identify the validity and applications of each source and find whether there are any gaps in the presented knowledge to research further. I will repeat this process until my supervisor and I are happy the full scope of the project has been thoroughly reviewed.
2. The implementation stage of the project will require a methodology that can be adapted to changing requirements or a shift in focus based on information learnt in the later stages of the research phase. I will use an agile-like methodology, of one to two week sprints determined by meetings with my supervisor, with any changes to requirements determined during these meetings with future sprints based on the previous.
3. I will use my framework to replicate the results of the researched papers, showcasing how to create new sampling methods and heuristics, and how to compare the results of different sampling, heuristics and models on different networks.
4. Finally, I will attempt to improve upon a sampling method or heuristic and use my framework to do so. This will demonstrate that my tool is applicable to real world research uses and potentially find something interesting or noteworthy.

E Timeline

The schedule is not fully decided at this point as the research phase is yet to be completed and the further points maybe change. Some points such as writing the report in the month leading to the deadline are certain, however the contents and direction of that report is not yet concrete. The purple bars on the gantt chart are the deliverables and the yellow bars are the main sub-tasks associated with those deliverables.

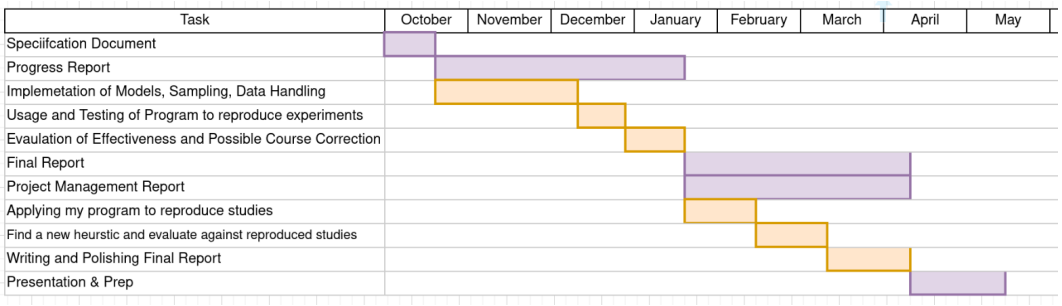


Figure 4: Gantt Chart of Expected Timeline

F Resources and Risks

F.1 Resources

1. Python and Standard Libraries - NetworkX, NumPy, Pandas, Matplotlib, Build tools. These are all commonly available on a variety of systems, are open source, and can be

built from source in case PyPI is unavailable.

2. Large Graph Datasets - Enron emails, airport traffic data, Gnutella data, Twitter tweet data and any other data sets as research progresses. The data is available from multiple sources on the internet, as well as being stored in the repository of my program under version control.

3. Batch Compute System - Department of Computer Science (DCS). The datasets I will be working with are large but not unmanageable on a typical desktop. The batch compute system will be used to learn and utilise good practice for scientific computing, as well as allowing for faster computation.

F.2 Risks

1. Failure to find a new heuristic or sampling method that is measurably better than those currently written about. In this case the project would still be a success as it would show that it allows for evaluation of heuristics effectively to be able to show that it is not statistically better.

2. Lost or damaged device containing code and documents. Using GitHub to be a remote mirror of local git repository and regularly committing code will prevent issues from lost work arising. DCS machines are more than adequate for processing needs.

G Legal, Social, Ethical and Professional Considerations

At this stage there are no legal, social, ethical or professional considerations to take into account. The project will not be using, processing or interacting with personal data or information.

References

[1] J. Archbold, S. Clohessy, D. Herath, N. Griffiths, and O. Oyebode. An agent-based model of the spread of behavioural risk-factors for cardiovascular disease in city-scale populations. *PLOS ONE*, 19(5):1–22, 05 2024.

[2] J. Archbold and N. Griffiths. Limiting concept spread in environments with interacting concepts. In *16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2017)*, pages 1332–1340. IFAAMAS, 2017.

[3] W. Chen, Y. Wang, and S. Yang. Efficient influence maximization in social networks. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 199–208, 2009.

[4] H. Franks, N. Griffiths, and S. S. Anand. Learning agent influence in mas with complex social networks. *Autonomous Agents and Multi-Agent Systems*, 28:836–866, 2014.

[5] M. Gjoka, M. Kurant, C. T. Butts, and A. Markopoulou. Practical recommendations on crawling online social networks. *IEEE Journal on Selected Areas in Communications*, 29(9):1872–1892, 2011.

[6] L. Jin, Y. Chen, P. Hui, C. Ding, T. Wang, A. Vasilakos, B. Deng, and X. Li. Albatross sampling: robust and effective hybrid vertex sampling for social graphs. 07 2011.

[7] D. Kempe, J. Kleinberg, and É. Tardos. Influential nodes in a diffusion model for social networks. In *international colloquium on automata, languages, and programming*, pages 1127–1138. Springer, 2005.

[8] C.-H. Lee, X. Xu, and D. Y. Eun. Beyond random walk and metropolis-hastings samplers: Why you should not backtrack for unbiased graph sampling, 2012.

[9] T. Wang, Y. Chen, Z. Zhang, T. Xu, L. Jin, P. Hui, B. Deng, and X. Li. Understanding graph sampling algorithms for social network analysis. *2011 31st International Conference on Distributed Computing Systems Workshops*, pages 123–128, 2011.

[10] W. Yang, L. Brenner, and A. Giua. Influence maximization in independent cascade networks based on activation probability computation. *IEEE Access*, 7:13745–13757, 2019.